Playwright (TypeScript/JavaScript) Interview Guide

Part 1: Conceptual questions with short answers Part 2: Coding scenarios with sample solutions (TypeScript)

PART 1: INTERVIEW QUESTIONS

A. Basics

1. **What is Playwright and how is it different from Selenium/Cypress?** Playwright is a Microsoft-maintained E2E framework that drives Chromium, Firefox and WebKit through one API. Differences: auto-waiting, built-in test runner, multi-tab/multi-context support, network interception, tracing, parallelism by default. Selenium uses WebDriver (slower, manual waits). Cypress runs inside the browser (limited multi-tab/cross-origin support).
2. **Which languages does Playwright support?** TypeScript/JavaScript, Python, Java, .NET.
3. **What is `@playwright/test`?** The built-in test runner: fixtures, parallel workers, retries, reporters, projects, assertions (`expect`), trace viewer.
4. **Difference between Browser, BrowserContext and Page?** Browser = the browser process. Context = isolated session (own cookies, storage), cheap to create. Page = a tab inside a context.
5. **How do you install and initialise a project?** `npm init playwright@latest`, then `npx playwright test`.
6. **What does `playwright.config.ts` contain?** `testDir`, `timeout`, `retries`, `workers`, `reporter`, `use` (baseURL, headless, trace, screenshot, video), `projects` (browsers/devices), `webServer`.

B. Locators and Waiting

7. **What are the recommended locators?** `getByRole`, `getByLabel`, `getByPlaceholder`, `getByText`, `getByTestId`. CSS/XPath only as a last resort.
8. **Locator vs ElementHandle?** Locators are lazy and re-resolve on every action (auto-retry, resilient to re-renders). ElementHandles point to a specific DOM node and can go stale. Prefer locators.
9. **How does auto-waiting work?** Before an action, Playwright checks the element is attached, visible, stable, enabled, and receives events. It retries until timeout.
10. **How do you handle dynamic elements?** Use web-first assertions (`await expect(locator).toBeVisible()`), `locator.waitFor()`, `filter({ hasText })`, `nth()`, chaining locators.
11. **Why avoid `waitForTimeout`?** It is a hard sleep, which makes tests slow and flaky. Use event/state-based waits.

12. **What is strict mode?** If a locator matches more than one element and you perform an action, Playwright throws. Fix with a more specific locator, `first()`, `nth()` or `filter`.
13. `waitForLoadState` **options?** `load`, `domcontentloaded`, `networkidle` (discouraged for most cases).

C. Assertions

14. **Web-first vs generic assertions?** `await expect(locator).toHaveText()` retries until timeout. `expect(await locator.textContent()).toBe()` does not retry, so it is flaky.
15. **What is soft assertion?** `expect.soft(...)` records failure but continues the test.
16. **Common matchers?** `toBeVisible`, `toHaveText`, `toContainText`, `toHaveValue`, `toHaveCount`, `toHaveURL`, `toHaveTitle`, `toBeChecked`, `toHaveScreenshot`, `toMatchSnapshot`.

D. Test Structure and Fixtures

17. **Hooks available?** `beforeAll`, `afterAll`, `beforeEach`, `afterEach`.
18. **What are fixtures?** Reusable, isolated setup/teardown units injected into tests via parameters (`page`, `context`, `request`, plus custom ones via `test.extend`).
19. **How do you run tests in parallel / serial?** Files run in parallel by default; `test.describe.configure({ mode: 'parallel' | 'serial' })`; `workers` in config.
20. **How do you tag and filter tests?** `@smoke` in title and `--grep @smoke`, or `test.describe` with tags, `--project`, `test.skip`, `test.fixme`, `test.only`.
21. **How do you do data-driven testing?** Loop over arrays and create a `test()` per row, or read JSON/CSV.

E. Page Object Model and Framework Design

22. **What is POM and why use it?** A class per page/component encapsulating locators and actions, so tests stay readable and changes happen in one place.
23. **How do you structure a framework?** `pages/`, `fixtures/`, `tests/`, `utils/`, `test-data/`, `config`, `.env` handling, reporters, CI config.
24. **How do you manage environments and secrets?** `dotenv`, `process.env`, per-environment config, CI secrets. Never commit credentials.

F. Authentication and State

25. **How do you avoid logging in for every test?** Log in once in a setup project, save `storageState`, reuse via `use.storageState`.
26. **How do you test multiple roles?** Multiple storage state files, one per role, with separate projects or contexts.

G. Network and API

27. **How do you mock an API?** `page.route('**/api/users', route => route.fulfill({ json: [...] }))`.
28. **How do you wait for a particular network call?** `page.waitForResponse(...)` or `waitForRequest`, started before the triggering action.
29. **How do you do API testing?** `request` fixture / `APIRequestContext` (`request.get/post`), assertions on status and body, combining API setup with UI verification.

H. Advanced UI Handling

30. **Frames?** `page.frameLocator('#iframe').getByRole(...)`.
31. **New tab/popup?** `const [popup] = await Promise.all([page.waitForEvent('popup'), click])`.
32. **Dialogs (alert/confirm)?** `page.on('dialog', d => d.accept())`, register before triggering.
33. **File upload/download?** `setInputFiles`, `waitForEvent('download')`, `download.saveAs`.
34. **Drag and drop, hover, keyboard?** `dragTo`, `hover`, `keyboard.press`.
35. **Shadow DOM?** Playwright CSS/role locators pierce open shadow DOM by default.
36. **Date pickers, autocompletes, infinite scroll, tables?** Combine locators, `filter`, loops, `mouse.wheel`, and wait for list size changes.

I. Debugging, Reporting, CI

37. **How do you debug?** `--debug`, Playwright Inspector, `page.pause()`, `--ui` mode, trace viewer (`trace: 'on-first-retry'`), VS Code extension, codegen (`npx playwright codegen`).
38. **Reporters?** `list`, `dot`, `line`, `html`, `json`, `junit`, `allure` (third-party).
39. **How do you run in CI?** Install browsers (`npx playwright install --with-deps`), run headless, upload `playwright-report` and traces as artifacts, shard with `--shard=1/4`.
40. **How do you reduce flaky tests?** Proper locators, web-first assertions, no hard waits, test isolation, controlled test data, mock unstable third parties, retries only as a safety net, trace analysis.
41. **Visual testing?** `toHaveScreenshot()` with thresholds and masked dynamic regions.
42. **Accessibility testing?** `@axe-core/playwright`.
43. **Mobile/responsive?** `devices['iPhone 13']` in projects, `viewport`, geolocation, permissions.

J. Typical Experience Questions

44. Biggest flaky test you fixed and how?
45. How did you decide what to automate vs keep manual?
46. How do you handle test data creation and cleanup?
47. How did you integrate Playwright with CI/CD and reporting?
48. How do you review a teammate's Playwright test?

PART 2: CODING SCENARIOS

Each scenario has the task and a sample solution.

Scenario 1: Basic login test

Task: Open the login page, log in with valid credentials, verify the dashboard heading and URL.

```ts
import { test, expect } from '@playwright/test';

test('valid user can log in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Password').fill('Password@123');
  await page.getByRole('button', { name: 'Sign in' }).click();

  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

Scenario 2: Page Object Model

Task: Refactor Scenario 1 into a POM.

```ts
```

```ts
// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly email: Locator;
  readonly password: Locator;
  readonly submit: Locator;
  readonly error: Locator;

  constructor(page: Page) {
    this.page = page;
    this.email = page.getByLabel('Email');
    this.password = page.getByLabel('Password');
    this.submit = page.getByRole('button', { name: 'Sign in' });
    this.error = page.getByRole('alert');
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.email.fill(email);
    await this.password.fill(password);
    await this.submit.click();
  }

  async expectError(text: string) {
    await expect(this.error).toContainText(text);
  }
}

// tests/login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test('invalid password shows error', async ({ page }) => {
  const login = new LoginPage(page);
  await login.goto();
  await login.login('user@example.com', 'wrong');
  await login.expectError('Invalid credentials');
});
```

Scenario 3: Custom fixture that injects a page object

Task: Provide `loginPage` as a fixture so tests don't create it manually.

```ts
// fixtures/index.ts
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

type MyFixtures = { loginPage: LoginPage };

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await use(loginPage);
  },
});
export { expect } from '@playwright/test';

// tests/login.spec.ts
import { test, expect } from '../fixtures';

test('login via fixture', async ({ loginPage, page }) => {
  await loginPage.login('user@example.com', 'Password@123');
  await expect(page).toHaveURL(/dashboard/);
});
```

Scenario 4: Login once, reuse storage state

Task: Authenticate in a setup step and reuse the session across tests.

```ts
// tests/auth.setup.ts
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.USER_EMAIL!);
  await page.getByLabel('Password').fill(process.env.USER_PASSWORD!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('**/dashboard');
  await page.context().storageState({ path: '.auth/user.json' });
});
```

```ts
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'], storageState: '.auth/user.json' },
      dependencies: ['setup'],
    },
  ],
});
```

Scenario 5: Data-driven tests

Task: Test login with several invalid inputs.

```ts
import { test, expect } from '@playwright/test';

const cases = [
  { name: 'empty email', email: '', password: 'abc', msg: 'Email is required' }
  { name: 'empty password', email: 'a@b.com', password: '', msg: 'Password is r
  { name: 'wrong password', email: 'a@b.com', password: 'bad', msg: 'Invalid cr
];

for (const c of cases) {
  test(`login fails: ${c.name}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(c.email);
    await page.getByLabel('Password').fill(c.password);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.getByRole('alert')).toContainText(c.msg);
  });
}
```

Scenario 6: Mock an API response

Task: Mock `/api/products` to return two products and verify the UI shows two rows. Then mock a 500 error and verify the error banner.

ts

```ts
import { test, expect } from '@playwright/test';

test('renders mocked products', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, name: 'Laptop', price: 900 },
        { id: 2, name: 'Mouse', price: 20 },
      ]),
    })
  );

  await page.goto('/products');
  await expect(page.getByRole('row')).toHaveCount(3); // header + 2
  await expect(page.getByText('Laptop')).toBeVisible();
});

test('shows error on 500', async ({ page }) => {
  await page.route('**/api/products', route => route.fulfill({ status: 500 }));
  await page.goto('/products');
  await expect(page.getByText('Something went wrong')).toBeVisible();
});
```

Scenario 7: Wait for a network response triggered by a click

Task: Click "Save" and assert the POST returns 201 with the right payload.

ts

```ts
import { test, expect } from '@playwright/test';

test('save triggers POST with 201', async ({ page }) => {
  await page.goto('/profile');
  await page.getByLabel('Name').fill('Asha');

  const [response] = await Promise.all([
    page.waitForResponse(r => r.url().includes('/api/profile') && r.request().m
    page.getByRole('button', { name: 'Save' }).click(),
  ]);

  expect(response.status()).toBe(201);
  const body = await response.json();
  expect(body.name).toBe('Asha');
});
```

Scenario 8: API test with the `request` fixture

Task: Create a user via API, fetch it, assert fields, delete it.

```ts
import { test, expect } from '@playwright/test';

test('user CRUD via API', async ({ request }) => {
  const create = await request.post('/api/users', {
    data: { name: 'Ravi', email: `ravi+${Date.now()}@example.com` },
  });
  expect(create.status()).toBe(201);
  const user = await create.json();

  const get = await request.get(`/api/users/${user.id}`);
  expect(get.ok()).toBeTruthy();
  expect(await get.json()).toMatchObject({ name: 'Ravi' });

  const del = await request.delete(`/api/users/${user.id}`);
  expect(del.status()).toBe(204);
});
```

Scenario 9: Create data via API, verify via UI

Task: Seed an order using the API, then confirm it appears in the UI.

```ts
```

```ts
import { test, expect } from '@playwright/test';

test('order created via API appears in UI', async ({ page, request }) => {
  const res = await request.post('/api/orders', { data: { item: 'Book', qty: 2
  const order = await res.json();

  await page.goto('/orders');
  const row = page.getByRole('row').filter({ hasText: String(order.id) });
  await expect(row).toContainText('Book');
  await expect(row).toContainText('2');
});
```

Scenario 10: Handle a new tab / popup

Task: Click a link that opens a new tab and assert its title.

ts

```ts
import { test, expect } from '@playwright/test';

test('opens help in new tab', async ({ page }) => {
  await page.goto('/');
  const [newPage] = await Promise.all([
    page.context().waitForEvent('page'),
    page.getByRole('link', { name: 'Help' }).click(),
  ]);
  await newPage.waitForLoadState();
  await expect(newPage).toHaveTitle(/Help Center/);
});
```

Scenario 11: iframe and JavaScript dialog

Task: Fill a field inside an iframe, and accept a confirm dialog.

ts

```ts
import { test, expect } from '@playwright/test';

test('iframe form', async ({ page }) => {
  await page.goto('/iframe-demo');
  const frame = page.frameLocator('#payment-frame');
  await frame.getByLabel('Card number').fill('4242424242424242');
  await expect(frame.getByLabel('Card number')).toHaveValue('4242424242424242')
});

test('accept confirm dialog', async ({ page }) => {
  await page.goto('/delete-demo');
  page.once('dialog', async dialog => {
    expect(dialog.message()).toContain('Are you sure');
    await dialog.accept();
  });
  await page.getByRole('button', { name: 'Delete' }).click();
  await expect(page.getByText('Deleted')).toBeVisible();
});
```

Scenario 12: File upload and download

ts

```ts
import { test, expect } from '@playwright/test';
import * as fs from 'fs';

test('upload file', async ({ page }) => {
  await page.goto('/upload');
  await page.setInputFiles('input[type=file]', 'test-data/sample.pdf');
  await expect(page.getByText('sample.pdf')).toBeVisible();
});

test('download report', async ({ page }) => {
  await page.goto('/reports');
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('button', { name: 'Export CSV' }).click(),
  ]);
  const path = `downloads/${download.suggestedFilename()}`;
  await download.saveAs(path);
  expect(fs.existsSync(path)).toBeTruthy();
  expect(fs.readFileSync(path, 'utf-8')).toContain('Name,Email');
});
```

Scenario 13: Web table. Find a row, read all values, sort check

Task: In an employee table, click "Edit" for "Priya" and verify the Salary column is sorted ascending after clicking the header.

```ts
import { test, expect } from '@playwright/test';

test('edit specific row', async ({ page }) => {
  await page.goto('/employees');
  const row = page.getByRole('row').filter({ hasText: 'Priya' });
  await row.getByRole('button', { name: 'Edit' }).click();
  await expect(page).toHaveURL(/edit/);
});

test('salary column sorts ascending', async ({ page }) => {
  await page.goto('/employees');
  await page.getByRole('columnheader', { name: 'Salary' }).click();

  const texts = await page.locator('table tbody tr td:nth-child(3)').allTextCon
  const nums = texts.map(t => Number(t.replace(/[^0-9.]/g, '')));
  const sorted = [...nums].sort((a, b) => a - b);
  expect(nums).toEqual(sorted);
});
```

Scenario 14: Autocomplete / dropdown

Task: Type "Pun" in a city search, pick "Pune" from suggestions.

```ts
```

```ts
import { test, expect } from '@playwright/test';

test('autocomplete select', async ({ page }) => {
  await page.goto('/search');
  await page.getByPlaceholder('City').fill('Pun');
  await page.getByRole('option', { name: 'Pune' }).click();
  await expect(page.getByPlaceholder('City')).toHaveValue('Pune');
});

test('native select', async ({ page }) => {
  await page.goto('/form');
  await page.getByLabel('Country').selectOption({ label: 'India' });
  await expect(page.getByLabel('Country')).toHaveValue('IN');
});
```

Scenario 15: Infinite scroll

Task: Scroll until at least 50 items are loaded.

ts

```ts
import { test, expect } from '@playwright/test';

test('infinite scroll loads 50+ items', async ({ page }) => {
  await page.goto('/feed');
  const items = page.locator('.feed-item');

  await expect(async () => {
    await page.mouse.wheel(0, 3000);
    expect(await items.count()).toBeGreaterThanOrEqual(50);
  }).toPass({ timeout: 20_000 });
});
```

Scenario 16: Multiple user roles in one test

Task: An admin approves a request that a normal user created. Use two isolated contexts.

ts

```ts
import { test, expect } from '@playwright/test';

test('admin approves user request', async ({ browser }) => {
  const userCtx = await browser.newContext({ storageState: '.auth/user.json' })
  const adminCtx = await browser.newContext({ storageState: '.auth/admin.json'
  const userPage = await userCtx.newPage();
  const adminPage = await adminCtx.newPage();

  await userPage.goto('/requests/new');
  await userPage.getByLabel('Title').fill('Laptop request');
  await userPage.getByRole('button', { name: 'Submit' }).click();

  await adminPage.goto('/admin/requests');
  await adminPage
    .getByRole('row')
    .filter({ hasText: 'Laptop request' })
    .getByRole('button', { name: 'Approve' })
    .click();

  await userPage.reload();
  await expect(userPage.getByText('Approved')).toBeVisible();

  await userCtx.close();
  await adminCtx.close();
});
```

Scenario 17: Visual comparison with masking

ts

```ts
import { test, expect } from '@playwright/test';

test('home page visual', async ({ page }) => {
  await page.goto('/');
  await expect(page).toHaveScreenshot('home.png', {
    mask: [page.locator('.timestamp'), page.locator('.ad-banner')],
    maxDiffPixelRatio: 0.01,
  });
});
```

Scenario 18: Accessibility scan

ts

```ts
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test('home page has no critical a11y violations', async ({ page }) => {
  await page.goto('/');
  const results = await new AxeBuilder({ page }).withTags(['wcag2a', 'wcag2aa']
  expect(results.violations).toEqual([]);
});
```

Scenario 19: Retry a flaky operation and soft assertions

ts

```ts
import { test, expect } from '@playwright/test';

test('dashboard widgets (soft checks)', async ({ page }) => {
  await page.goto('/dashboard');
  await expect.soft(page.getByTestId('revenue')).toBeVisible();
  await expect.soft(page.getByTestId('orders')).toBeVisible();
  await expect.soft(page.getByTestId('users')).toBeVisible();
});

test('poll until status becomes Done', async ({ page }) => {
  await page.goto('/jobs/123');
  await expect(async () => {
    await page.reload();
    await expect(page.getByTestId('status')).toHaveText('Done');
  }).toPass({ intervals: [1000, 2000, 5000], timeout: 30_000 });
});
```

Scenario 20: Mobile emulation and geolocation

ts

```ts
import { test, expect, devices } from '@playwright/test';

test.use({
  ...devices['iPhone 13'],
  geolocation: { latitude: 18.5204, longitude: 73.8567 },
  permissions: ['geolocation'],
});

test('shows nearby stores on mobile', async ({ page }) => {
  await page.goto('/stores');
  await expect(page.getByRole('button', { name: 'Menu' })).toBeVisible();
  await expect(page.getByText('Pune')).toBeVisible();
});
```

Scenario 21: Full config example (CI-friendly)

ts

```ts
import { defineConfig, devices } from '@playwright/test';
import dotenv from 'dotenv';
dotenv.config();

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  expect: { timeout: 5_000 },
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined,
  reporter: [['html', { open: 'never' }], ['junit', { outputFile: 'results.xml'
  use: {
    baseURL: process.env.BASE_URL || 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

PART 3: LIVE-CODING PRACTICE TASKS (no solutions, try them yourself)

1. Automate search + filter + add to cart + checkout on a demo e-commerce site (e.g. saucedemo.com) using POM and fixtures.
2. Log in once with storage state, then verify a protected page for two different roles.
3. Intercept an API call and change the response to simulate an empty state and an error state.
4. Write a test that verifies a sortable, paginated table across all pages.
5. Write a reusable helper that retries a flaky action up to 3 times with logging.
6. Add custom reporting: attach screenshots and API responses to the HTML report with `testInfo.attach`.
7. Run tests in 3 shards in a CI pipeline and merge the HTML reports.

QUICK TIPS FOR THE INTERVIEW

- Always prefer `getByRole` / `getByTestId` and say why.
- Mention web-first assertions and no hard waits.
- Talk about test isolation, independent data, and cleanup.
- Explain how you would debug a failure: trace viewer, then screenshots/video, then logs.
- Be ready to explain how you reduced flakiness and run time in a real project.